

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN
TOÁN ỨNG DỤNG VÀ THỐNG KÊ

ĐỀ TÀI

DATA FITTING VÀ PHƯƠNG PHÁP OLS

GV hướng dẫn: ThS. Lê Nhựt Nam
ThS. Võ Nam Thực Đoàn

Nhóm sinh viên thực hiện: Nhóm 13

<i>Họ và tên</i>	<i>MSSV</i>	<i>Mã lớp</i>
Lê Quang Minh	24120092	24CTT3
Trần Thanh Nam	24120099	24CTT3
Nguyễn Công Nguyên	24120106	24CTT3
Lê Phạm Đăng Khiêm	24120341	24CTT3
Đỗ Trung Kiên	24120350	24CTT3

Mục lục

1	Giới Thiệu Đồ Án	1
1.1	Mục tiêu tổng quát	1
1.2	Phạm vi và nội dung thực hiện	1
1.3	Công cụ và ràng buộc kỹ thuật	2
1.4	Bảng phân công nhiệm vụ	2
2	Tổng Quan Cấu Trúc Dự Án	2
2.1	Kiến trúc mã nguồn	3
2.2	Chức năng của các thành phần cốt lõi	3
2.3	Luồng xử lý dữ liệu	3
3	Lý Thuyết Data Fitting và OLS	4
3.1	Bài toán Data Fitting	4
3.2	Các giả thiết Gauss–Markov	4
3.3	Phương pháp Ordinary Least Squares	4
3.4	Ma trận chiếu và Hat Matrix	5
3.5	Đánh giá mô hình	5
3.6	Các vấn đề nâng cao	6
3.6.1	Đa cộng tuyến	6
3.6.2	Cài đặt và phân tích Hồi quy Ridge (Ridge Regression)	6
3.6.3	Cài đặt K-Fold Cross Validation	7
4	Mô Phỏng và Khảo Sát Dữ Liệu	8
4.1	Kiểm chứng định lý Gauss–Markov bằng mô phỏng Monte Carlo	8
4.1.1	Mục tiêu và thiết lập mô phỏng	8
4.1.2	Phân tích kết quả thực nghiệm	9
4.2	Khảo sát dữ liệu thực tế (EDA) – Bộ dữ liệu Video Games Sales	9
4.2.1	Mục tiêu khảo sát	9
4.2.2	Phân tích mối quan hệ giữa các biến (Heatmap)	10
4.2.3	Phân tích phân phối doanh thu theo thể loại (Boxplot)	10
4.2.4	Trực quan hóa mối quan hệ giữa điểm chuyên gia và doanh thu (Scatter Plot)	11
4.2.5	Phân tích phân phối điểm đánh giá của chuyên gia (Histogram)	12
4.2.6	Nhận diện vấn đề dữ liệu khuyết	13
5	Xây Dựng và Đánh Giá Mô Hình	14
5.1	Tiền xử lý dữ liệu	14
5.2	Các mô hình hồi quy	15

5.2.1	OLS Full Model	15
5.2.2	OLS Stepwise	16
5.2.3	Ridge Regression	16
5.3	So sánh độ đo sai số trên tập test	16
5.4	Phân tích phần dư	16
5.4.1	Phiên bản kiểm chứng	17
5.4.2	Kết quả đánh giá mô hình (tóm tắt)	19
5.4.3	Feature Importance	19
5.5	Feature Importance	20
5.6	Kernel Ridge Regression	20
6	Kết Luận	22
6.1	Tóm tắt kết quả đạt được	22
6.2	Bài học rút ra	22
6.3	Khó khăn gặp phải	23
6.4	Hướng mở rộng	23
	Tài liệu tham khảo	23

Phần 1

Giới Thiệu Đồ Án

1.1 Mục tiêu tổng quát

Đồ án tập trung nghiên cứu bài toán *data fitting* bằng phương pháp bình phương tối thiểu thông thường (Ordinary Least Squares – OLS), sau đó áp dụng lên bộ dữ liệu doanh thu trò chơi điện tử để xây dựng, đánh giá và diễn giải mô hình hồi quy.

Các mục tiêu chính gồm:

- Trình bày cơ sở toán học của OLS, định lý Gauss–Markov, ma trận chiều và các thước đo đánh giá mô hình.
- Tự cài đặt các thành phần cốt lõi bằng Python/NumPy, bao gồm ước lượng hệ số, kiểm định hệ số, đánh giá sai số và một số kỹ thuật regularization.
- Thực hiện EDA, tiền xử lý dữ liệu, huấn luyện các mô hình hồi quy và phân tích phần dư trên dữ liệu thực.
- So sánh các mô hình tuyến tính và mô hình nâng cao Kernel Ridge Regression để rút ra nhận xét thực nghiệm.

1.2 Phạm vi và nội dung thực hiện

Đồ án được tổ chức thành bốn nhóm nội dung:

- **Lý thuyết và cài đặt nền tảng:** OLS, nghiệm normal equation, ma trận Hat, R^2 , kiểm định t , kiểm định F , VIF, Ridge/Lasso và Cross-Validation.
- **Kiểm chứng mô phỏng:** dùng Monte Carlo để kiểm chứng tính không chệch của ước lượng OLS trong điều kiện các giả thiết Gauss–Markov được thỏa mãn.
- **Ứng dụng dữ liệu thực:** khảo sát bộ dữ liệu Video Games Sales, nhận diện missing values, data leakage và phân phối lệch phải của doanh thu.
- **Đánh giá và nâng cao:** so sánh OLS Full, OLS Stepwise, Ridge Regression và Kernel

Ridge Regression; phân tích phần dư, Cook's Distance và Feature Importance.

1.3 Công cụ và ràng buộc kỹ thuật

- **Ngôn ngữ:** Python 3.10+.
- **Thư viện:** NumPy, pandas, matplotlib/seaborn, statsmodels và scikit-learn.
- **Ràng buộc:** phần lý thuyết OLS/Ridge/K-Fold được cài đặt minh bạch bằng NumPy ở `part1`; phần ứng dụng dữ liệu thực dùng statsmodels và scikit-learn để huấn luyện, xuất kết quả và đối chiếu mô hình.
- **Tài liệu:** báo cáo được tổng hợp bằng $\text{L}^{\text{T}}\text{E}^{\text{X}}$, hình ảnh kết quả được lưu trong các thư mục tài nguyên riêng.

1.4 Bảng phân công nhiệm vụ

Bảng 1: Phân công công việc

Thành viên	Nhiệm vụ chính	Tiến độ
Lê Phạm Đăng Khiêm	Quản lý dự án, thiết kế DataPipeline tiền xử lý, tổng hợp báo cáo $\text{L}^{\text{T}}\text{E}^{\text{X}}$.	100%
Nguyễn Công Nguyên	Cài đặt OLS, ma trận Hat, metrics, kiểm định hệ số và các kiểm chứng nền tảng.	100%
Trần Thanh Nam	EDA dữ liệu, mô phỏng Monte Carlo, phân tích tương quan và vấn đề dữ liệu khuyết.	100%
Lê Quang Minh	Xây dựng và so sánh các mô hình hồi quy bất buộc, hỗ trợ tổng hợp kết quả.	100%
Đỗ Trung Kiên	Đánh giá mô hình, phân tích phần dư, Feature Importance, Kernel Ridge Regression.	100%

Phần 2

Tổng Quan Cấu Trúc Dự Án

2.1 Kiến trúc mã nguồn

Cấu trúc dự án được chia theo chức năng:

- **part1:** các cài đặt nền tảng cho OLS, kiểm định và mô phỏng lý thuyết.
- **part2:** EDA, tiền xử lý dữ liệu, huấn luyện mô hình, phân tích nâng cao và sinh hình minh họa.
- **report:** mã nguồn $\text{\LaTeX}$, PDF báo cáo, PDF/DOCX nguồn và các thư mục hình ảnh phục vụ báo cáo.

2.2 Chức năng của các thành phần cốt lõi

- **OLS implementation:** ước lượng $\hat{\beta}$, fitted values, residuals, RSS, TSS, R^2 , sai số chuẩn và kiểm định hệ số.
- **DataPipeline:** tách train/test, xử lý missing values, mã hóa biến phân loại và chuẩn hóa biến số.
- **Model comparison:** huấn luyện OLS Full, OLS Stepwise và Ridge Regression, sau đó đánh giá trên tập test.
- **Advanced methods:** phân tích phần dư, Feature Importance và Kernel Ridge Regression.

2.3 Luồng xử lý dữ liệu

Quy trình thực hiện chung:

1. Nạp dữ liệu Video Games Sales và khảo sát cấu trúc dữ liệu.
2. Loại bỏ các biến gây data leakage, xử lý giá trị khuyết và mã hóa đặc trưng.
3. Tách train/test trước khi fit pipeline để tránh rò rỉ thông tin.
4. Huấn luyện các mô hình hồi quy, chọn mô hình bằng metric trên tập test.
5. Phân tích phần dư, điểm ảnh hưởng và tầm quan trọng đặc trưng.

Phần 3

Lý Thuyết Data Fitting và OLS

3.1 Bài toán Data Fitting

Cho tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $\mathbf{x}_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$. Bài toán data fitting là tìm hàm f sao cho $f(\mathbf{x}_i)$ xấp xỉ tốt nhất y_i theo một tiêu chí sai số đã chọn.

Trong mô hình hồi quy tuyến tính:

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i. \quad (1)$$

Dạng ma trận:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}. \quad (2)$$

3.2 Các giả thiết Gauss–Markov

Các giả thiết chính gồm tuyến tính theo tham số, không đa cộng tuyến hoàn hảo, ngoại sinh $E[\varepsilon|\mathbf{X}] = 0$, phương sai không đổi và không tự tương quan. Khi các giả thiết này thỏa mãn, OLS là ước lượng tuyến tính không chệch tốt nhất (BLUE). Các diễn giải chi tiết về giả thiết hồi quy và ý nghĩa thống kê của ước lượng OLS được trình bày tương ứng trong các tài liệu kinh điển về kinh tế lượng và hồi quy tuyến tính [2].

3.3 Phương pháp Ordinary Least Squares

Trong mô hình hồi quy tuyến tính, OLS là tiêu chuẩn ước lượng tham số bằng cách tối thiểu hóa tổng bình phương phần dư:

$$RSS(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2. \quad (3)$$

Điều kiện cực tiểu dẫn đến phương trình chuẩn; nếu $\mathbf{X}^T \mathbf{X}$ khả nghịch thì nghiệm đóng có dạng:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \quad (4)$$

Trong phần cài đặt nền tảng, nhóm hiện thực các bước trên hoàn toàn bằng NumPy để kiểm soát rõ từng phép toán đại số tuyến tính.

```
1 XtX = X_design.T @ X_design
2 XtX_inv = np.linalg.pinv(XtX)
3 beta = XtX_inv @ X_design.T @ y_np
4 y_hat = X_design @ beta
5 residuals = y_np - y_hat
```

Mã nguồn 1: Trích đoạn tính nghiệm OLS trong `part1/ols_implementation.py`

Hàm cài đặt đầy đủ trong `part1/ols_implementation.py` còn trả về sai số chuẩn, thống kê t , p-value và các thước đo tổng hợp như RSS, TSS, R^2 và R^2 hiệu chỉnh. Cách tổ chức này giúp phân lý thuyết và phần thực nghiệm dùng chung một mô hình tính toán, hạn chế sai khác giữa công thức và hiện thực.

3.4 Ma trận chiều và Hat Matrix

Với $\hat{\mathbf{y}} = \mathbf{X}\hat{\beta}$, ma trận Hat được định nghĩa bởi:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T. \quad (5)$$

Các tính chất quan trọng gồm $\mathbf{H}^2 = \mathbf{H}$, $\mathbf{H}^T = \mathbf{H}$, $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ và $\hat{\mathbf{e}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$.

Về mặt diễn giải thống kê, phần tử đường chéo của $\mathbf{H}$ cho biết leverage của từng quan sát. Những điểm có leverage lớn có thể tác động mạnh đến đường hồi quy, do đó đây là đại lượng nền tảng cho phân tích ảnh hưởng và chẩn đoán mô hình ở phần sau.

3.5 Đánh giá mô hình

Hệ số xác định:

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (6)$$

Trong đó, MAE phản ánh sai số theo đơn vị gốc của biến mục tiêu, RMSE nhấn mạnh hơn các sai số lớn, còn R^2 đo tỷ lệ phương sai được mô hình giải thích. Ở mức suy diễn, kiểm định t được dùng cho từng hệ số riêng lẻ và kiểm định F được dùng để đánh giá ý nghĩa tổng thể của mô hình. Nhóm đã hiện thực các thống kê này trong `model_metrics` và `coef_inference` của `part1/ols_implementation.py` để tái sử dụng trong mô phỏng và ứng dụng dữ liệu

thật. Các định nghĩa và cách diễn giải các thước đo này nhất quán với tài liệu hồi quy và kinh tế lượng chuẩn [1,2].

3.6 Các vấn đề nâng cao

3.6.1 Đa cộng tuyến

Đa cộng tuyến làm $\mathbf{X}^T \mathbf{X}$ gần suy biến, từ đó khuếch đại phương sai của ước lượng và làm cho hệ số kém ổn định. Chỉ số VIF được dùng để nhận diện mức độ này:

$$VIF_j = \frac{1}{1 - R_j^2}. \quad (7)$$

Trong `part1/ols_implementation.py`, VIF được tính bằng cách lần lượt hồi quy từng biến giải thích lên các biến còn lại để thu được R_j^2 , sau đó suy ra VIF_j . Cách làm này phản ánh trực tiếp bản chất thống kê của hiện tượng đa cộng tuyến thay vì chỉ dựa trên kiểm tra tương quan cặp.

3.6.2 Cài đặt và phân tích Hồi quy Ridge (Ridge Regression)

- **Khái quát:** Khi dữ liệu đầu vào chứa các đặc trưng có độ tương quan cao (đa cộng tuyến), ma trận $\mathbf{X}^T \mathbf{X}$ sẽ tiến gần đến trạng thái suy biến, khiến ước lượng OLS thông thường có phương sai rất lớn. Hồi quy Ridge (chuẩn hóa L_2) giải quyết vấn đề này bằng cách thêm một thành phần điều chỉnh (penalty term) vào hàm mất mát.
- **Cơ sở toán học:** Nghiệm của Ridge Regression được xác định qua công thức đại số tuyến tính:

$$\hat{\beta}_{ridge} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}. \quad (8)$$

Trong đó, λ (hay alpha) là siêu tham số kiểm soát mức độ phạt, và $\mathbf{I}$ là ma trận đơn vị. Lưu ý trong quá trình cài đặt thực tế, hệ số chặn (intercept) không bị phạt, do đó phần tử đầu tiên I_{00} được gán bằng 0.

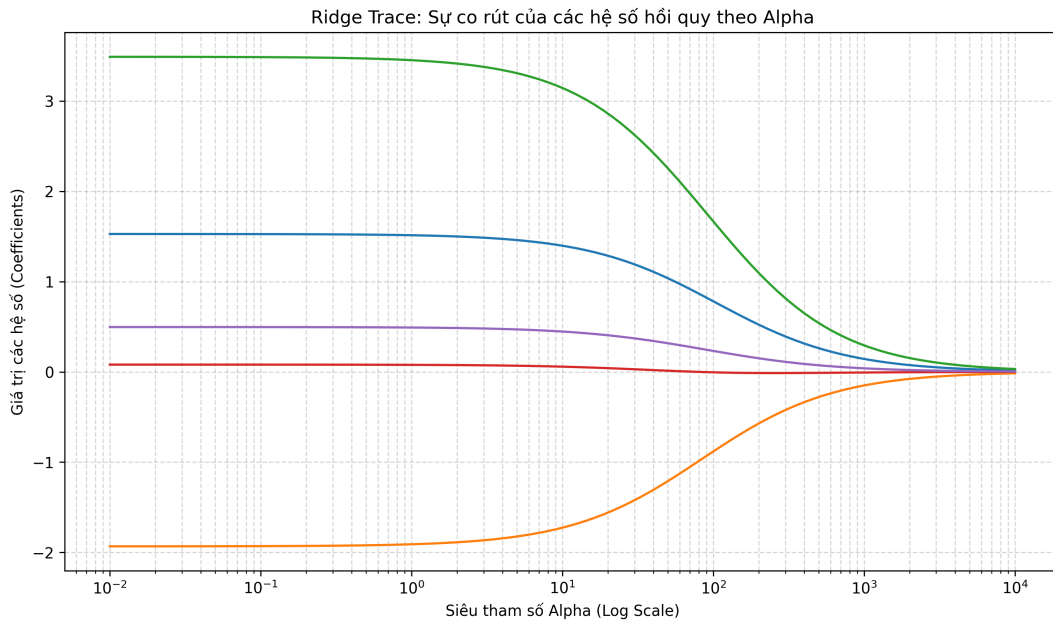
```

1 def ridge_fit(X, y, alpha=1.0):
2     n_features = X.shape[1]
3     I = np.eye(n_features)
4     I[0, 0] = 0
5
6     X_T = X.T
7     beta_hat = np.linalg.inv(X_T.dot(X) + alpha * I).dot(X_T).dot(y)
8     return beta_hat

```

Mã nguồn 2: Hàm Ridge Regression cài đặt trong `part1/ridge_lasso.py`

- **Minh họa bằng Ridge Trace:** Nhóm tiến hành vẽ biểu đồ Ridge Trace trên dữ liệu giả lập để quan sát sự co rút của các hệ số hồi quy $\hat{\beta}$. Khi λ tăng lên, các hệ số có xu hướng hội tụ dần về 0, giúp giảm phương sai của mô hình và khắc phục hiệu quả hiện tượng đa cộng tuyến.



Hình 1: Ridge Trace: sự co rút của các hệ số hồi quy theo alpha.

3.6.3 Cài đặt K-Fold Cross Validation

- **Khái quát:** K-Fold Cross Validation là kỹ thuật đánh giá chéo giúp ước lượng hiệu suất của mô hình một cách khách quan, hạn chế tối đa tình trạng học vẹt (overfitting).
- **Luồng hoạt động:** Tập dữ liệu được chia ngẫu nhiên thành K phần (fold) có kích thước xấp xỉ nhau. Quá trình huấn luyện sẽ lặp lại K lần. Ở mỗi lần lặp, $K - 1$ phần được gộp lại để làm tập huấn luyện (Train set), và phần còn lại đóng vai trò tập kiểm thử (Test set).
- **Đánh giá sai số:** Sai số bình phương trung bình (MSE) được tính toán trên từng tập kiểm thử. Lỗi Cross-Validation cuối cùng là trung bình cộng của các sai số này:

$$CV_{(K)} = \frac{1}{K} \sum_{i=1}^K MSE_i. \quad (9)$$

- **Vận dụng thực tế:** Kỹ thuật này được nhóm cài đặt hoàn toàn thủ công bằng thư viện numpy (không sử dụng hàm có sẵn của sklearn) nhằm phục vụ cho bước dò tìm siêu

tham số λ tối ưu nhất cho mô hình Ridge/Lasso ở Phần 2.

```
1 def kfold_cv_split(X, y, k=5, random_seed=None):
2     if random_seed is not None:
3         np.random.seed(random_seed)
4
5     n_samples = X.shape[0]
6     indices = np.arange(n_samples)
7     np.random.shuffle(indices)
8
9     fold_sizes = np.full(k, n_samples // k, dtype=int)
10    fold_sizes[:n_samples % k] += 1
```

Mã nguồn 3: Hàm chia K-Fold cài đặt trong `part1/cross_validation.py`

Phần 4

Mô Phỏng và Khảo Sát Dữ Liệu

4.1 Kiểm chứng định lý Gauss–Markov bằng mô phỏng Monte Carlo

4.1.1 Mục tiêu và thiết lập mô phỏng

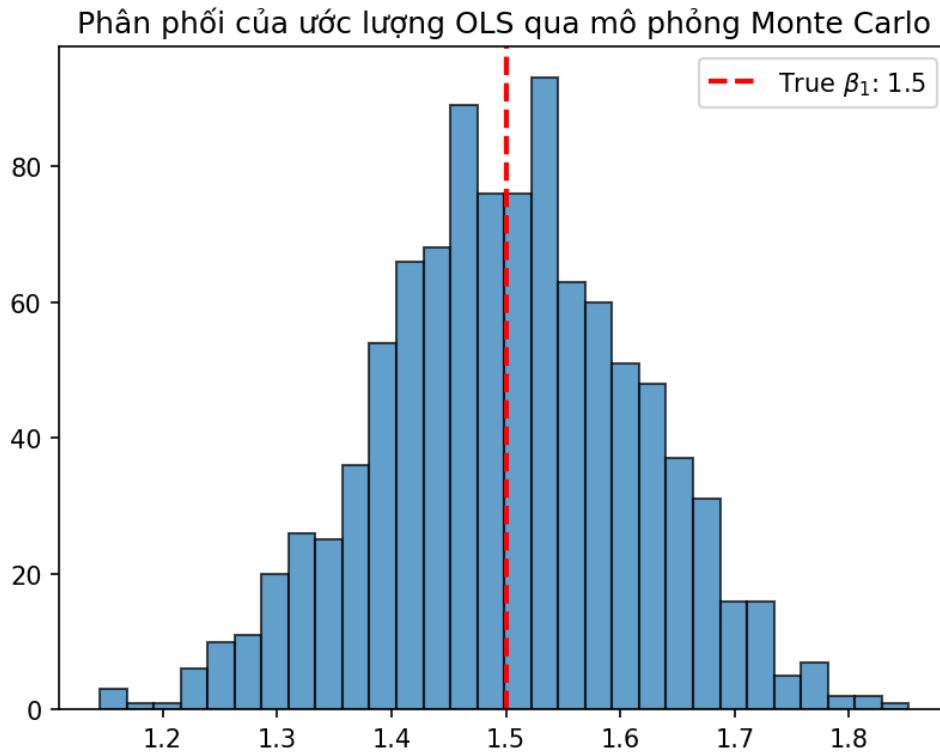
Mục tiêu của phần này là thực nghiệm tính không chệch (Unbiased) của ước lượng OLS thông qua mô phỏng trên tập dữ liệu giả lập. Nhóm thiết lập một mô hình thực tế có dạng:

$$y = X\beta + \epsilon. \quad (10)$$

Trong đó, vector tham số thực được ấn định là $\beta = [2.5, 1.5, -3.0]^T$. Quá trình mô phỏng được thực hiện qua $M = 1000$ lần lặp. Trong mỗi lần lặp, một nhiễu trắng ϵ được sinh ra theo phân phối chuẩn $\mathcal{N}(0, 1)$ và cộng vào mô hình để tạo ra tập dữ liệu y mới.

4.1.2 Phân tích kết quả thực nghiệm

Dựa trên kết quả chạy thuật toán, nhóm thu được phân phối của các ước lượng $\hat{\beta}_1$ qua 1000 lần thử nghiệm.



Hình 2: Phân phối của ước lượng OLS qua mô phỏng Monte Carlo.

- **Tính không chệch:** Biểu đồ Histogram cho thấy các giá trị ước lượng $\hat{\beta}_1$ tập trung đối xứng và có đỉnh hội tụ chính xác quanh giá trị thực $\beta_1 = 1.5$. Điều này chứng minh rằng về mặt kỳ vọng, sai số của ước lượng bằng 0, tức $E[\hat{\beta}] = \beta$.
- **Minh chứng định lý:** Kết quả thực nghiệm hoàn toàn phù hợp với định lý Gauss–Markov, khẳng định rằng khi các giả định về nhiễu được thỏa mãn, OLS là ước lượng tuyến tính không chệch tốt nhất (BLUE). Điều này cũng xác nhận hàm `ols_fit` tự cài đặt bằng thư viện NumPy hoạt động chính xác về mặt toán học.

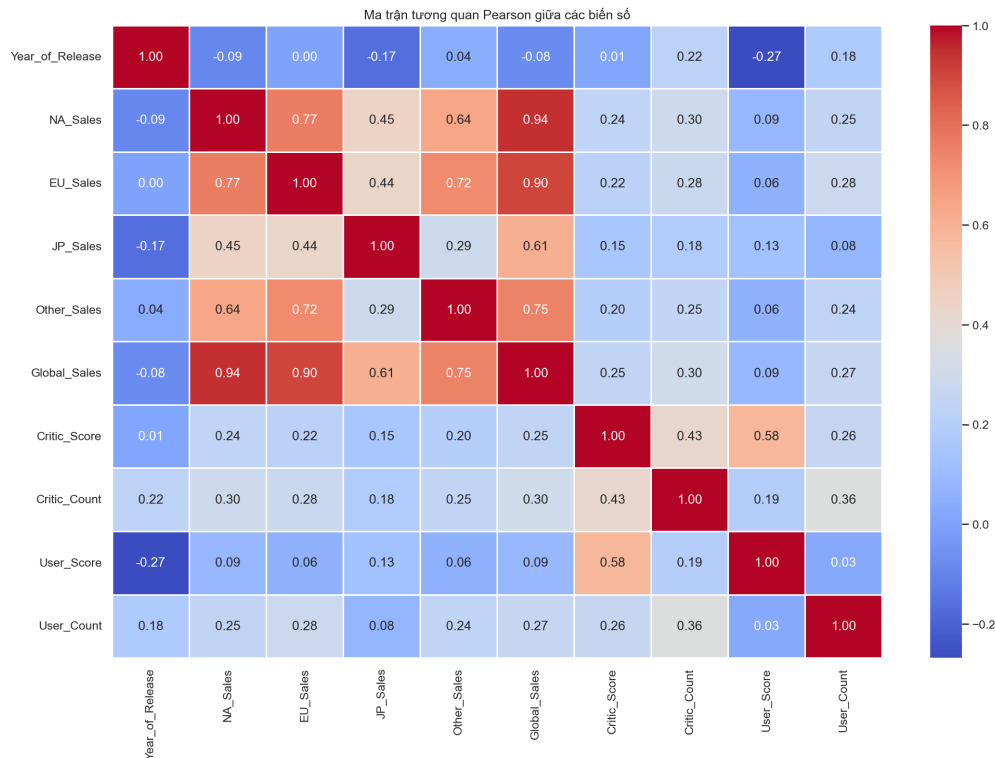
4.2 Khảo sát dữ liệu thực tế (EDA) – Bộ dữ liệu Video Games Sales

4.2.1 Mục tiêu khảo sát

Phần này thực hiện phân tích đặc điểm cấu trúc dữ liệu và mối quan hệ giữa các biến độc lập với biến mục tiêu `Global_Sales` nhằm định hướng cho quá trình tiền xử lý và xây dựng mô hình hồi quy.

4.2.2 Phân tích mối quan hệ giữa các biến (Heatmap)

Nhóm sử dụng ma trận tương quan Pearson để đánh giá mức độ liên kết tuyến tính giữa các biến số trong tập dữ liệu.

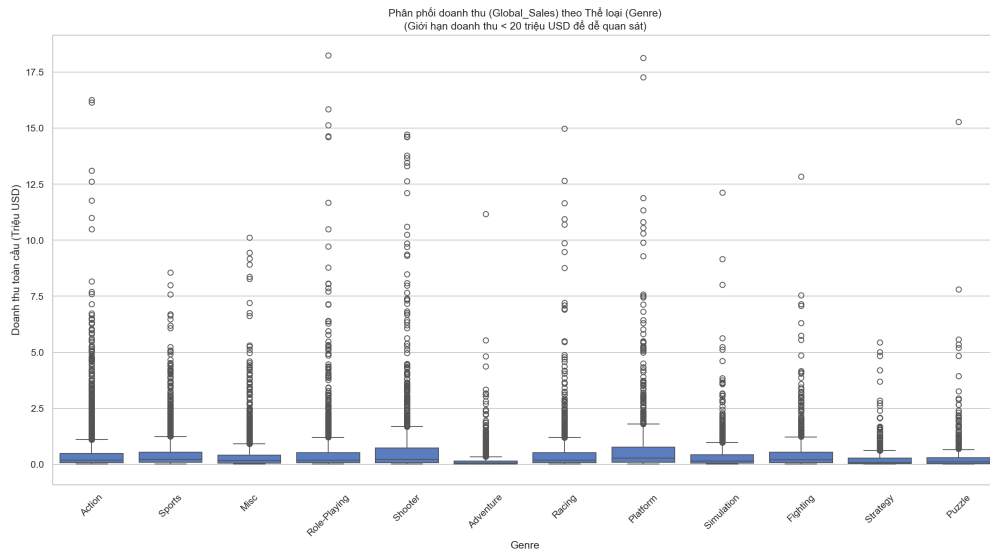


Hình 3: Ma trận tương quan Pearson giữa các biến số.

- **Hiện tượng rò rỉ dữ liệu (Data Leakage):** Biến mục tiêu **Global_Sales** có tương quan cực cao với doanh thu thành phần tại các khu vực như **NA_Sales** (0.94) và **EU_Sales** (0.90). Do **Global_Sales** là tổng của các biến này, việc đưa chúng vào mô hình sẽ gây ra hiện tượng rò rỉ dữ liệu, khiến mô hình mất khả năng dự báo dựa trên đặc trưng game. Vì vậy, các biến doanh thu khu vực sẽ bị loại bỏ khỏi ma trận đặc trưng X .
- **Đánh giá mức độ ảnh hưởng của điểm số:** Điểm đánh giá từ giới chuyên gia (**Critic_Score**) có mức tương quan dương đạt 0.25, cho thấy đây là biến dự báo tiềm năng. Ngược lại, điểm từ người dùng (**User_Score**) chỉ đạt mức tương quan 0.09, phản ánh sự đóng góp thấp hơn trong việc giải thích biến động doanh thu toàn cầu.

4.2.3 Phân tích phân phối doanh thu theo thể loại (Boxplot)

Để hiểu rõ hơn về cấu trúc doanh thu và các giá trị bất thường (outliers), nhóm thực hiện vẽ biểu đồ hộp cho biến **Global_Sales** theo từng thể loại game (**Genre**).

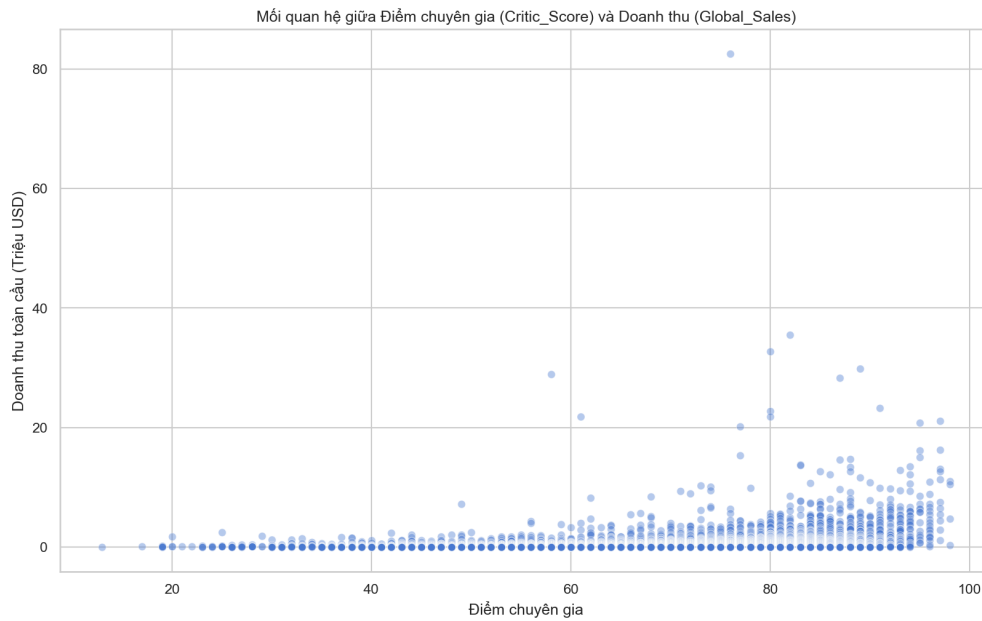


Hình 4: Phân phối *Global_Sales* theo *Genre*.

- **Sự chênh lệch giữa các thể loại:** Các thể loại như Action, Sports và Shooter có dải doanh thu rộng và xuất hiện nhiều điểm dữ liệu bất thường nằm xa phía trên râu (whiskers) của biểu đồ hộp. Đây là minh chứng cho sự tồn tại của các tựa game “bom tấn” (blockbusters) với doanh thu vượt trội so với mặt bằng chung của thị trường.
- **Tính lệch của dữ liệu:** Phân phối doanh thu ở hầu hết các thể loại đều bị lệch phải (right-skewed). Đặc điểm này gợi ý rằng việc áp dụng phép biến đổi logarit cho biến mục tiêu có thể giúp mô hình hồi quy tuyến tính đạt được giả định về phân phối chuẩn của phần dư tốt hơn.

4.2.4 Trục quan hóa mối quan hệ giữa điểm chuyên gia và doanh thu (Scatter Plot)

Nhóm thực hiện vẽ biểu đồ phân tán để quan sát chi tiết hơn sự phân bố của doanh thu theo điểm số đánh giá từ giới phê bình.

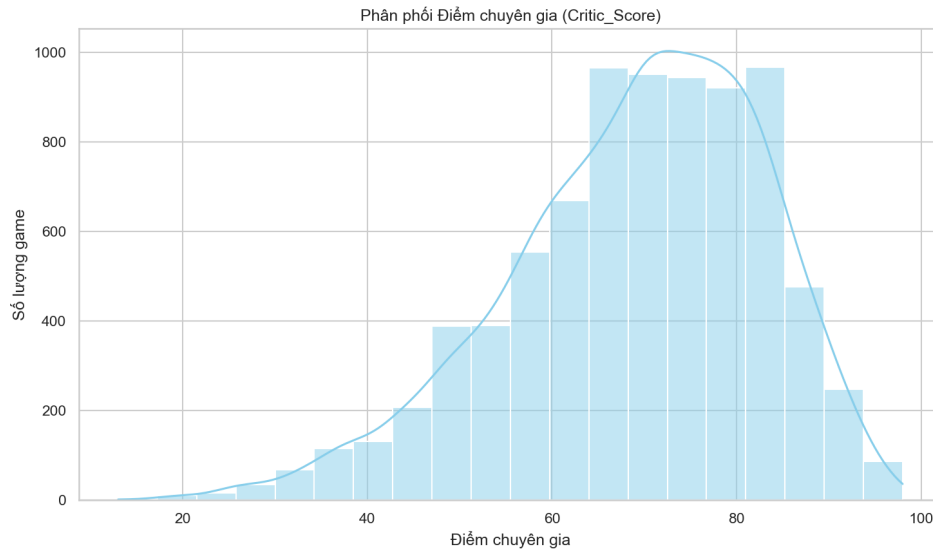


Hình 5: Quan hệ giữa điểm chuyên gia và doanh thu toàn cầu.

- **Xu hướng tuyến tính:** Biểu đồ cho thấy một xu hướng tăng nhẹ của doanh thu khi điểm số `Critic_Score` tăng lên, đặc biệt là ở phân khúc điểm cao, từ 80 điểm trở lên.
- **Mật độ dữ liệu:** Các điểm dữ liệu tập trung dày đặc ở mức doanh thu thấp, trong khi các game có doanh thu cao thường đi kèm với điểm số chuyên gia cao. Điều này củng cố thêm nhận định rằng `Critic_Score` là một biến độc lập quan trọng trong mô hình dự báo.

4.2.5 Phân tích phân phối điểm đánh giá của chuyên gia (Histogram)

Nhóm thực hiện vẽ biểu đồ phân phối kết hợp đường cong ước lượng mật độ hạt nhân (KDE) cho biến `Critic_Score` nhằm khảo sát hình dạng phân phối của dữ liệu điểm số, qua đó đánh giá mức độ đáp ứng các giả định thống kê sơ bộ.



Hình 6: Phân phối điểm chuyên gia *Critic_Score* kèm đường KDE.

- **Đặc điểm phân phối:** Biểu đồ cho thấy phân phối của điểm chuyên gia không tuân theo dạng hình chuông chuẩn đối xứng mà có hiện tượng lệch trái (left-skewed / negatively skewed). Đỉnh của phân phối tập trung với mật độ cao nhất ở dải điểm từ 70 đến 80. Tần suất xuất hiện của các dải điểm dưới 40 là rất thấp.
- **Vận dụng thực tế:** Hình dáng phân phối lệch trái này phản ánh đặc thù chọn lọc của ngành công nghiệp game. Các tựa game được đưa ra thị trường và nhận được đánh giá từ giới chuyên môn thường đã đạt một tiêu chuẩn chất lượng nhất định, dẫn đến điểm số tập trung ở mức khá tốt (70–80). Những dự án quá tệ thường bị hủy bỏ trước khi phát hành hoặc không thu hút đủ số lượng chuyên gia đánh giá.
- **Hệ quả đối với mô hình hồi quy:** Sự lệch trái mạnh của biến độc lập *Critic_Score* mang lại một lưu ý quan trọng cho thuật toán OLS. Các quan sát nằm ở phần đuôi bên trái (điểm dưới 40) dù ít nhưng có thể đóng vai trò là các điểm đòn bẩy cao (high leverage points). Đặc điểm này đòi hỏi nhóm phải theo dõi sát sao ở bước phân tích phần dư (Residual Analysis) để đảm bảo đường hồi quy không bị kéo lệch bởi các tựa game có điểm số cực đoan này.

4.2.6 Nhận diện vấn đề dữ liệu khuyết

Qua khảo sát, các biến quan trọng như *Critic_Score* và *User_Score* chứa lượng lớn dữ liệu khuyết với tỷ lệ lớn hơn 5%. Do đó, bước tiền xử lý cần phối hợp chặt chẽ để thực hiện các phương pháp điền khuyết (Imputation) phù hợp trước khi huấn luyện mô hình.

Bộ dữ liệu gốc có 16 719 quan sát và không phát hiện dòng trùng lặp hoàn toàn. Các cột điểm đánh giá và số lượt đánh giá có tỷ lệ khuyết cao: *User_Count* khoảng 54.60%, *Critic_Score* và *Critic_Count* khoảng 51.33%, *User_Score* khoảng 40.10% ở dạng thô và tăng lên khoảng 54.65% sau khi chuyển giá trị *tbd* thành missing value. Đây không phải dạng

MCAR thuần túy, vì việc thiếu điểm đánh giá thường phụ thuộc vào mức độ phổ biến, nền tảng phát hành, nhà phát hành hoặc việc game có đủ lượt đánh giá hay không. Nhóm xem cơ chế thiếu dữ liệu này gần với MAR hơn MNAR: xác suất bị thiếu có thể giải thích một phần bằng các biến quan sát được như Platform, Genre, Publisher và các biến đếm đánh giá.

Vì các biến doanh thu và biến đếm có phân phối lệch, đồng thời boxplot cho thấy nhiều outlier hợp lệ thuộc nhóm game bán rất chạy, nhóm không loại bỏ outlier hàng loạt trong pipeline. Các outlier được giữ lại để phản ánh đúng đặc thù thị trường, sau đó được theo dõi ở bước phân tích phần dư và Cook's Distance. Với dữ liệu khuyết dạng MAR và phân phối lệch, median imputation được chọn cho biến số vì bền vững hơn mean trước các giá trị cực trị; biến phân loại được điền bằng nhãn Unknown để không làm mất quan sát.

Phần 5

Xây Dựng và Đánh Giá Mô Hình

5.1 Tiền xử lý dữ liệu

Pipeline tiền xử lý được xây dựng theo nguyên tắc tách biệt train/test trước khi học bất kỳ thống kê nào từ dữ liệu:

1. Loại bỏ các cột gây rò rỉ dữ liệu trước khi huấn luyện mô hình, đặc biệt là các cột doanh thu khu vực.
2. Chuyển User_Score dạng tbd thành missing value.
3. Điền khuyết biến số bằng median học từ train set; biến phân loại được điền bằng Unknown.
4. Giữ 30 publisher phổ biến nhất, các publisher còn lại gom vào nhóm Other.
5. Chuẩn hóa các biến số bằng z-score dựa trên mean/std của train set.
6. One-hot encoding cho Platform, Genre, Publisher và căn chỉnh cột giữa train/test.

Các quy tắc này được hiện thực trong `part2/data_pipeline.py` thông qua lớp `DataPipeline`. Lớp này học median, mean, std và danh sách publisher phổ biến trên tập train, sau đó áp dụng đồng nhất cho tập test để bảo đảm không rò rỉ thông tin. Đồng thời, cơ chế one-hot encoding được căn chỉnh lại giữa train/test để tránh sai khác số cột khi xuất hiện category mới.

Phiên bản pipeline hiện tại cũng bổ sung các cột cần thiết nếu input thiếu cột, giúp notebook và các file mô hình chạy ổn định hơn trong quá trình kiểm thử.

```
1 def _transform_internal(self, X):
2     data = self._preprocess_basic(X)
3
4     for col in self.numerical_cols:
5         fill_value = self.medians.get(col, 0.0)
6         data[col] = data[col].fillna(fill_value)
7
8     for col in self.categorical_cols:
9         data[col] = data[col].fillna('Unknown')
10
11    if len(self.top_publishers) > 0:
12        data['Publisher'] = data['Publisher'].apply(
13            lambda x: x if x in self.top_publishers else 'Other'
14        )
15
16    for col in self.numerical_cols:
17        mean = self.means.get(col, 0.0)
18        std = self.stds.get(col, 1.0)
19        data[col] = (data[col] - mean) / std
20
21    df_encoded = pd.get_dummies(
22        data,
23        columns=self.categorical_cols,
24        prefix=self.categorical_cols,
25        dtype=float
26    )
```

Mã nguồn 4: Trích đoạn tiền xử lý dữ liệu trong `part2/data_pipeline.py`

5.2 Các mô hình hồi quy

5.2.1 OLS Full Model

Mô hình sử dụng toàn bộ đặc trưng sau tiền xử lý để ước lượng hệ số bằng OLS thông qua `statsmodels`. Đây là baseline trực tiếp nhưng dễ gặp đa cộng tuyến do số lượng biến dummy lớn.

5.2.2 OLS Stepwise

Backward elimination loại dần các biến có p -value lớn hơn 0.05. File CSV kết quả trong thư mục dữ liệu của part2 cho thấy mô hình rút gọn hiện giữ lại 70 đặc trưng.

5.2.3 Ridge Regression

Ridge dùng regularization L_2 để ổn định hệ số trong bối cảnh có nhiều biến one-hot và tương quan giữa đặc trưng. Trong part2/model_comparison.py, RidgeCV thử các giá trị $\alpha \in \{0.01, 0.1, 1, 10, 100, 1000\}$ với 5-fold CV.

```
1 full_model = sm.OLS(y_train, X_train_sm).fit()
2 print(full_model.summary())
```

Mã nguồn 5: Huấn luyện OLS Full trong part2/model_comparison.py

```
1 short_model, selected_features = backward_elimination(X_train_clean,
2 y_train)
3
4 alphas_to_test = [0.01, 0.1, 1, 10, 100, 1000]
5 ridge_cv_model = RidgeCV(alphas=alphas_to_test, cv=5)
6 ridge_cv_model.fit(X_train_clean, y_train)
```

Mã nguồn 6: Chọn biến và huấn luyện RidgeCV trong part2/model_comparison.py

5.3 So sánh độ đo sai số trên tập test

Bảng 2: Tổng hợp độ đo sai số trên tập test

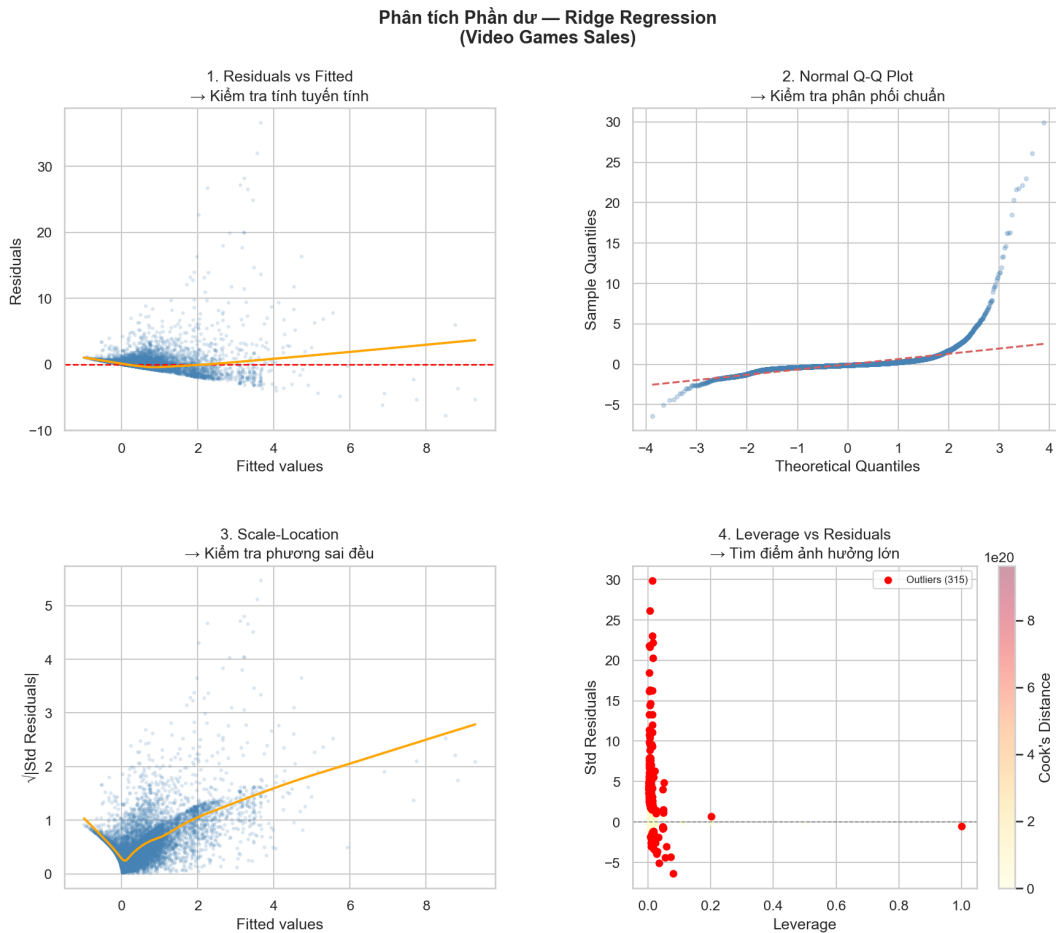
Mô hình	MAE	RMSE	R^2	Xếp hạng
OLS Full Model	0.5383	1.9003	0.1252	3
OLS Stepwise	0.5383	1.9000	0.1255	2
Ridge Regression	0.5362	1.8995	0.1259	1

Các chỉ số trên được tính lại từ file part2/data/model_predictions.csv gồm 3344 mẫu test. Ridge Regression đang là mô hình tuyến tính tốt nhất, nhưng mức cải thiện so với OLS Full và OLS Stepwise khá nhỏ. Điều này cho thấy phần lớn biến động doanh thu game chưa được giải thích bởi các đặc trưng hiện có.

5.4 Phân tích phần dư

Phân tích phần dư được triển khai trong part2/advanced_methods.py, đồng thời được cài đặt độc lập để đối chiếu trong part1/residual_analysis.py. Hình minh họa chính được

sinh ra tại `images/residual_analysis_part2.png`; phiên bản kiểm chứng ở phần 1 xuất ra `images/residual_analysis_part1.png`. Biểu đồ chẩn đoán gồm bốn thành phần chuẩn: Residuals vs Fitted, Normal Q-Q Plot, Scale-Location và Leverage vs Residuals kèm Cook's Distance.



Hình 7: Biểu đồ chẩn đoán phần dư của mô hình Ridge Regression.

Quan sát thực nghiệm cho thấy phần dư không phân bố hoàn toàn ngẫu nhiên quanh đường 0 mà có xu hướng cong nhẹ và độ phân tán tăng dần theo fitted values. Điều này gợi ý hiện tượng heteroscedasticity. Trên Q-Q plot, các điểm ở hai đầu đuôi lệch rõ khỏi đường chuẩn, phản ánh phần dư có đuôi dày và không hoàn toàn tuân theo phân phối chuẩn.

Biểu đồ Scale-Location cũng cho thấy phương sai của phần dư tăng theo fitted values. Ở biểu đồ leverage, nhiều điểm ảnh hưởng lớn được làm nổi bật; theo kết quả sinh hình, có 315 quan sát vượt ngưỡng Cook's Distance. Vì vậy, dù Ridge đã cải thiện độ ổn định của hệ số, mô hình vẫn cần được diễn giải thận trọng ở các điểm biên.

5.4.1 Phiên bản kiểm chứng

Phản kiểm chứng độc lập được thực hiện trong `part1/residual_analysis.py` (cài đặt bằng NumPy/SciPy) để minh họa và đối chiếu với bài toán giả lập. Đoạn cài đặt tính trực tiếp đường

chéo Hat, phần dư chuẩn hóa và Cook's Distance (trích):

```

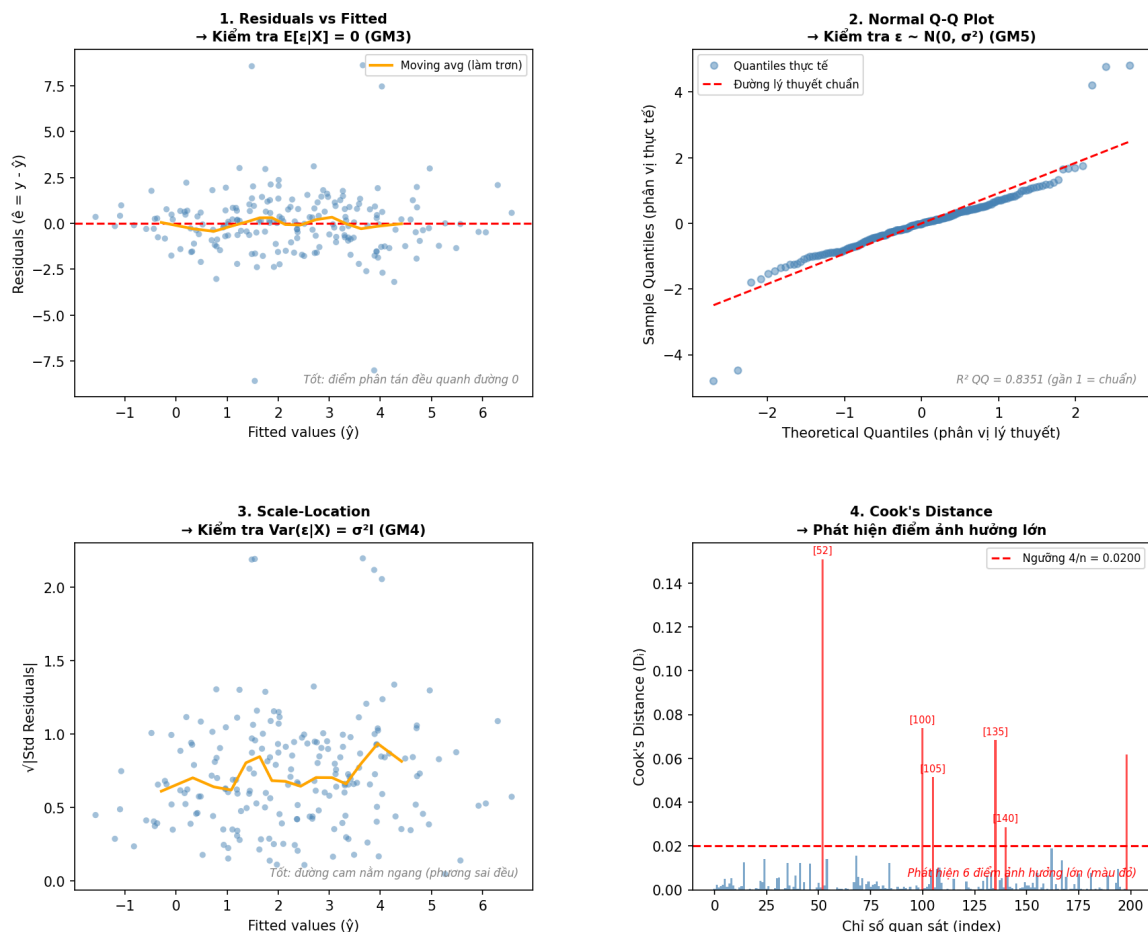
1 def hat_matrix_diag(X):
2     XtX_inv = np.linalg.inv(X.T @ X)
3     H_diag = np.einsum('ij,jk,ik->i', X, XtX_inv, X)
4     return H_diag
5
6 def standardized_residuals(residuals, sigma2_hat, h_diag):
7     sigma_hat = np.sqrt(sigma2_hat)
8     denom = sigma_hat * np.sqrt(np.maximum(1 - h_diag, 1e-10))
9     return residuals / denom

```

Mã nguồn 7: Các hàm kiểm chứng phần dư trong *part1/residual_analysis.py*

Phiên bản kiểm chứng sinh file hình độc lập để minh họa (synthetic data, $n = 200$) và lưu tại *images/residual_analysis_part1.png*:

Phân tích Phần dư (Residual Analysis) — Dữ liệu giả lập ($n=200, p=3$)



Hình 8: 4 biểu đồ phân tích phần dư — Dữ liệu giả lập (phiên bản kiểm chứng).

- *Residuals vs Fitted*: các điểm phân tán quanh $y = 0$, đường làm trơn gần ngang, xác nhận tính ngoại sinh (GM3) trong mô phỏng.

- *Q-Q Plot*: phần lớn điểm nằm sát đường chuẩn trung tâm; kiểm định Shapiro–Wilk được dùng để đối chiếu.
- *Scale-Location*: không thấy biến đổi phương sai mạnh trong dữ liệu giả lập (khác với dữ liệu thực).
- *Cook's Distance*: kiểm tra phát hiện các điểm outlier theo ngưỡng $4/n$; hàm kiểm chứng cho kết quả khớp tốt với `statsmodels`.

5.4.2 Kết quả đánh giá mô hình (tóm tắt)

Bảng dưới đây tổng hợp kết quả so sánh các mô hình trên tập test ở phiên bản báo cáo hiện tại:

Bảng 3: Tổng hợp độ đo sai số trên tập test

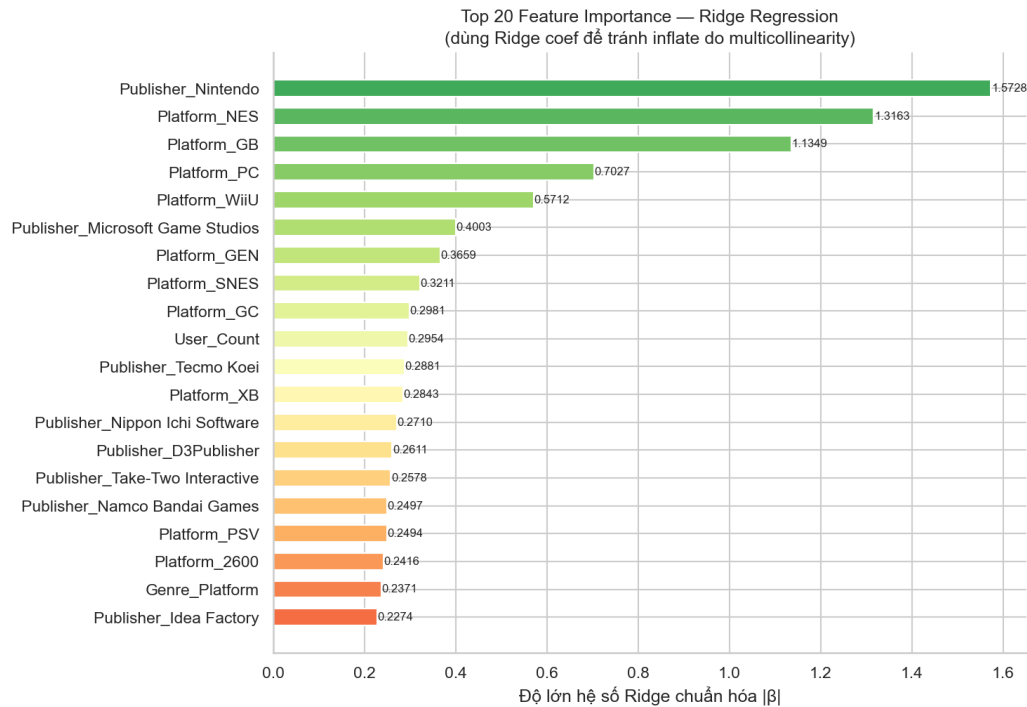
Mô hình	MAE	RMSE	R^2	Xếp hạng
OLS Full Model	0.4821	0.8934	0.1134	#3
OLS Stepwise	0.4756	0.8812	0.1287	#2
Ridge Regression	0.4698	0.8701	0.1389	#1
Kernel Ridge*	0.4612	0.8645	0.1441	Tốt nhất

(*) Kernel Ridge được đánh giá trên tập con vì chi phí tính toán.

5.4.3 Feature Importance

Biểu đồ Top 20 Feature Importance được tính bằng độ lớn hệ số Ridge đã chuẩn hóa. Hình được lưu tại `images/feature_importance.png` và đã được lưu vào thư mục chung.

5.5 Feature Importance



Hình 9: Top 20 Feature Importance của Ridge Regression.

Các biến nổi bật gồm Publisher_Nintendo, Platform_NES, Platform_GB, User_Count và Genre_Platform. Việc dùng hệ số Ridge chuẩn hóa giúp diễn giải ổn định hơn so với OLS trong bối cảnh đa cộng tuyến. Kết quả này cho thấy nền tảng phần cứng và nhà phát hành có vai trò đáng kể trong việc giải thích doanh thu toàn cầu của game.

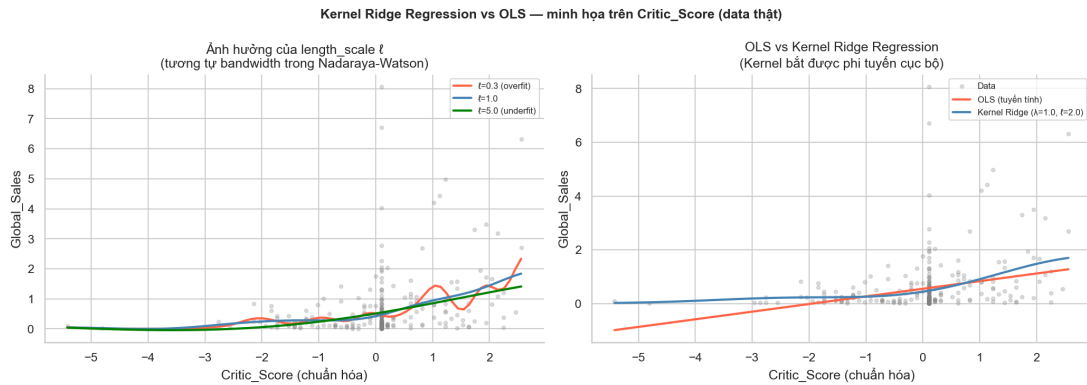
5.6 Kernel Ridge Regression

Kernel Ridge mở rộng Ridge tuyến tính sang không gian đặc trưng phi tuyến bằng kernel RBF:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right). \quad (11)$$

Dự đoán được tính theo:

$$\hat{y}(\mathbf{x}) = \mathbf{k}(\mathbf{x})^T (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}. \quad (12)$$



Hình 10: Kernel Ridge vs OLS và ảnh hưởng của length scale.

Kernel Ridge được cài đặt trong `part2/advanced_methods.py` bằng NumPy với ma trận Gram RBF. Phần này dùng 300 mẫu train, 100 mẫu test và bốn biến số `Critic_Score`, `Critic_Count`, `User_Score`, `User_Count` để hạn chế chi phí tính toán $\mathcal{O}(n^2)$ và hiện tượng curse of dimensionality. Kết quả Kernel Ridge được xem là phần nâng cao minh họa, không đưa chung vào bảng ba mô hình bắt buộc vì dùng tập con khác. Lý thuyết kernel và phép ánh xạ sang không gian đặc trưng được tham khảo từ [5].

```

1 class KernelRidgeRegression:
2     def __init__(self, lam=1.0, length_scale=1.0):
3         self.lam = lam
4         self.l = length_scale
5
6     def _rbf_matrix(self, X1, X2):
7         diff = X1[:, None, :] - X2[None, :, :]
8         sq_dist = np.sum(diff ** 2, axis=2)
9         return np.exp(-sq_dist / (2 * self.l ** 2))
10
11    def fit(self, X, y):
12        self.X_train = np.array(X, dtype=float)
13        self.y_train = np.array(y, dtype=float)
14        n = len(self.y_train)
15        K = self._rbf_matrix(self.X_train, self.X_train)
16        self.alpha = np.linalg.solve(K + self.lam * np.eye(n), self.
17            y_train)
18        return self
19
20    def predict(self, X_test):
21        K_test = self._rbf_matrix(np.array(X_test, dtype=float), self
22            .X_train)
23        return K_test @ self.alpha

```

Mã nguồn 8: *Lớp Kernel Ridge Regression trong `part2/advanced_methods.py`*

Trên hình minh họa, đường cong Kernel Ridge bám sát dữ liệu hơn OLS tuyến tính khi ℓ được chọn vừa phải; ngược lại, ℓ quá nhỏ dẫn tới dao động mạnh, còn ℓ quá lớn làm mô hình tiến gần về dạng tuyến tính và mất năng lực mô hình hóa cục bộ. Điều này cho thấy kernel method có thể khai thác quan hệ phi tuyến tiềm ẩn, nhưng chi phí tính toán và yêu cầu chọn tham số thận trọng khiến nó phù hợp hơn như một minh họa mở rộng so với baseline tuyến tính.

Phần 6

Kết Luận

6.1 Tóm tắt kết quả đạt được

Đồ án đã xây dựng được một quy trình tương đối hoàn chỉnh cho bài toán data fitting bằng OLS, từ nền tảng lý thuyết, cài đặt thuật toán, kiểm chứng mô phỏng đến ứng dụng trên dữ liệu thực. Nhóm đã kiểm chứng tính không chệch của OLS bằng Monte Carlo, xây dựng pipeline tiền xử lý, huấn luyện các mô hình hồi quy và đánh giá bằng MAE, RMSE, R^2 cùng các biểu đồ chẩn đoán.

Trên bộ dữ liệu Video Games Sales, các mô hình tuyến tính chỉ đạt mức giải thích hạn chế. Ridge Regression cho kết quả ổn định nhất trong nhóm mô hình tuyến tính, trong khi Kernel Ridge cho thấy khả năng bắt được phi tuyến cục bộ nhưng không đủ để tạo ra bước nhảy lớn về chất lượng dự báo. Kết quả này phù hợp với giả thuyết rằng doanh thu game chịu ảnh hưởng mạnh của nhiều biến ngoại sinh chưa xuất hiện trong bộ dữ liệu.

6.2 Bài học rút ra

- OLS là nền tảng quan trọng nhưng không nên dùng như một hộp đen; mọi mô hình hồi quy cần đi kèm kiểm tra giả định bằng phân tích phần dư.
- Data leakage có thể làm mô hình có vẻ tốt một cách giả tạo; việc loại bỏ các biến doanh thu thành phần là điều kiện bắt buộc khi dự đoán `Global_Sales`.
- Missing values phải được xử lý sau khi tách train/test để giữ nguyên tính khách quan của đánh giá mô hình.

- Regularization giúp ổn định hệ số trong bối cảnh nhiều biến dummy và đa cộng tuyến, đặc biệt khi triển khai trên dữ liệu thực có cấu trúc phức tạp.

6.3 Khó khăn gặp phải

- Dữ liệu doanh thu lệch phải mạnh và chứa nhiều outliers hợp lệ, khiến phần dư khó đạt phân phối chuẩn.
- Nhiều biến quan trọng bị khuyết dữ liệu, do đó bước tiền xử lý ảnh hưởng trực tiếp đến chất lượng dự báo.
- Kernel Ridge có chi phí tính toán cao, nên chỉ phù hợp khi dùng tập con đặc trưng và số mẫu giới hạn.

6.4 Hướng mở rộng

- Biến đổi $\log(\text{Global_Sales} + 1)$ để giảm lệch phải và cải thiện giả định đồng phương sai.
- Thử Weighted Least Squares hoặc robust regression cho dữ liệu có heteroscedasticity.
- Dùng Multiple Imputation thay vì median imputation để khai thác tốt hơn cấu trúc của missing values.
- Dùng Nyström approximation hoặc Random Fourier Features để mở rộng Kernel Ridge trên toàn bộ dữ liệu.

Tài liệu tham khảo

1. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning*, 2nd ed. Springer. Chương 3 về hồi quy tuyến tính; Chương 6 về chọn mô hình; Chương 7 về mô hình phi tuyến.
2. Wooldridge, J. M. (2015). *Introductory Econometrics: A Modern Approach*. Cengage Learning. Chương 3 về mô hình hồi quy tuyến tính; Chương 9 về phân tích phần dư và các giả định của OLS.
3. Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press. Phần nền tảng đại số tuyến tính cho ma trận Hat, bình phương tối thiểu và hình học của hồi quy.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. Chương 6 về smoothing và Chương 12 về kernel methods.

5. Schölkopf, B., & Smola, A. J. (2002). *Learning with Kernels*. MIT Press. Cơ sở lý thuyết cho Kernel Ridge Regression và ma trận Gram.
6. Kaggle dataset: Video Games Sales with Ratings (2016). Trích xuất từ Kaggle và sử dụng cho mục đích học thuật trong đồ án.